

Multi-scale graph attention subspace clustering network

Tong Wang^a, Junhua Wu^{a,*}, Zhenquan Zhang^b, Wen Zhou^a, Guang Chen^c, Shasha Liu^d

^a School of Computer Science and Technology, Nanjing Tech University, Jiangsu 211816, China

^b School of Computer Science and Engineering, Southeast University, Jiangsu 210096, China

^c School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing 100876, China

^d School of Computer Science, Beijing University of Posts and Telecommunications, Beijing 100876, China

ARTICLE INFO

Article history:

Received 4 February 2021

Revised 14 June 2021

Accepted 19 June 2021

Available online 23 June 2021

Communicated by Zidong Wang

Keywords:

Unsupervised learning

Node clustering

Graph convolutional networks

ABSTRACT

Deep subspace clustering (DSC) network which uses deep auto-encoders (DAE) mapping raw data into a latent space for clustering has achieved significant performance. However, when it comes to graph-based clustering, it fails to encode the node attribute and graph structure simultaneously, which degrades the clustering performance. Meanwhile, existing graph convolutional network (GCN) based clustering methods usually do not have a clustering-oriented objection function since the process of learning representation and clustering are separated. In addition, both aforementioned methods can neither catch the multi-scale information nor use the current clustering labels effectively, which lead to the suboptimal performance. To this end, we proposed a novel end-to-end framework called Multi-Scale Graph Attention Subspace Clustering Network (MSGAN). By employing a novel GCN-based feature extraction module, it can effectively capture the node representation on graph-based datasets. Moreover, a multi-scale self-expression module is designed for obtaining a more discriminative coefficient representation from each layer of the encoder and a self-supervised module is introduced for supervising the learning of node representation. Specifically, we train and optimize them in a unified framework so that different modules can benefit from each other. Extensive experiments demonstrate the superiority of our method compared with several state-of-the-art methods.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Graph-based clustering [1,2] is one of the most important part of clustering tasks, the goal of which is to segment attributed graph into several disjoint clusters satisfying the nodes in the same cluster should more likely have similar features. However, traditional methods might face some difficulties while analyzing graph structured data because they only consider a shallow relation between nodes.

Since the rapid development of deep learning has made considerable achievement, the idea of deep clustering which utilizes the powerful representation ability of deep learning for clustering has drawn widespread attention. Significant efforts [3–14,15] have been proposed in order to learn a more discriminative representation for clustering. Among them, auto-encoders (AE) [16] has gradually become famous by capturing complex relationships from original features through stacked non-linear layers. Based on this, deep subspace clustering network (DSC) [17] has made a success,

which introduces self-expression layer to construct an affinity matrix for clustering.

However, conventional auto-encoders are not able to fully exploit node relations in graph structured data. To tackle this issue, numerous graph convolutional networks (GCNs) have been presented. Among them, variational auto-encoders (VGAE) [18] made an early attempt using a graph convolutional network (GCN) [19] encoder and a inner-product decoder learning both graph structure and node attributes for node representation. For learning a more robust node representation, adversarially regularized variational graph auto-encoders (ARVGAE) [20] are proposed by incorporating an adversarial method of VGAE. Nevertheless, the node representation of VGAE and ARVGAE cannot be learned perfectly since both VGAE and ARVGAE use GCN approach, which only reconstructs the graph structure without the node attribute in the learning process. To this end, marginalized graph auto-encoders (MGAE) [21] have been proposed by stacking single layer GAE using linear activation function and marginalization process. Despite achieving superior performance compared with many state-of-the-art methods, MGAE completely neglects the information of the graph structure embedded in latent representation since it do not employ any hidden layers for reconstructing the node attribute. Hence, MGAE

* Corresponding author.

E-mail address: wujh@njtech.edu.cn (J. Wu).

cannot describe the dimension of latent representation and make a linear mapping, which degrades the clustering performance. Recently, a deep model called graph attention auto-encoders (GATE) [22] has been proposed, which has symmetric deep graph auto-encoders in both encoding and decoding process for the reconstruction of node representation and utilizes the attention mechanism improving the learning of node relations. Though effectively encoded the node representation, actually, GATE does not employ a clustering-oriented loss function but simply using K-means for the clustering tasks, which leads to the suboptimal performance.

In fact, in spite of achieving impressive performance for node clustering, the aforementioned methods still exist the following problems need to be overcome. (1) They merely utilize the representation of the deepest hidden layer of the encoder for clustering tasks which wastes plenty of useful feature information embedded in other layers of the encoder. (2) They fail to utilize the current clustering label to supervise the learning of node representation, so the learned latent node representation can not obtain the useful clustering information. (3) They do not consider an end-to-end framework since the process of learning node representation and clustering are separated.

Motivated by the above observations, we propose a novel end-to-end framework for graph-based clustering called Multi-Scale Graph Attention Subspace Clustering Network (MSGAN). We employ the feature extraction module to extract the latent node representation from the graph structured data. For fully utilizing the embedded features embedded in all the encoder layers, we develop the multi-scale self-expression module to obtain the individual coefficient representation from different layers of the encoder and then fuse them into a final coefficient representation with an adaptive method. Besides, we employ the deep subspace clustering module for obtaining the clustering labels with a clustering-oriented loss function. Finally, We design the self-supervised module, which utilizes the clustering label as a supervisor to guide the learning of the node representation. All the modules are jointly trained and optimized in a unified framework so that they can benefit from each other and help the node representation learn more clustering-oriented information. Fig. 1 shows the illustration of our proposed model. To sum up, the key contributions of this paper is summarized as follows.

- We develop a multi-scale self-expression module to fully exploit the relation of node representations in all the layers of the encoder, which helps utilize the embedded multi-scale information.
- We introduce a self-supervised module to effectively guide the learning of the node representation. Thus, the learned node representation would contain clustering information, which will greatly help obtain a better clustering performance.
- Extensive experiments on three graph datasets and two large-scale real-world datasets show the superiority of our MSGAN against other state-of-the-art methods.

2. Related work

The past decade saw an upsurge of subspace clustering and many researches made some attempts. These attempts can be concluded into solving two sub-problems. The first is to learn a affinity matrix Λ [23] from the original data in which Λ_{ij} is the similarity between the i^{th} and the j^{th} sample, the second is to apply spectral clustering methods on the affinity matrix to find the segmentation of data. Compared these two sub-problems, learning a discriminative affinity matrix is more important. To this end, numerous studies on learning an affinity matrix have been proposed and can be

simply concluded into three categories: factorization based [24,25], model based [26–28] and self-expression based approach [29,31,5,31]. Some significant work has been made such as Wang et al. [27] utilize a non-negative weight matrix for block diagonal representation learning to constrain the learning of affinity matrix. Yin et al. [28] attempt to obtain an adaptive low-rank graph affinity matrix by combining low-rank representation and graph construction.

To date, the self-expression based approaches have become the most popular because of its robustness and the lower computation complexity. In practice, however, most of the self-expression based approaches rely on the linear subspace assumption which is not tenable in practical problems. A few works based on a pre-defined kernel matrix [32–34] which still have some problems hard to be explained. To address the limitation of the linear assumption, deep subspace clustering network (DSC) [17] has been proposed leading a subspace-friendly latent space. Furthermore, DSC network using the novel self-expression coefficient which is a fully-connected layer without any activation and bias achieving joint training that can lead property data into auto-encoder (AE). Since DSC has shown encouraging performance, many other deep subspace clustering methods [35,36] based on AE recently have been proposed. For example, Huang et al. propose deep clustering via weighted k-subspace network (DWSC) [36] by optimizing AE and weighted k-subspace network in a unified framework. However, we observe that these AE-based subspace clustering method may suffer from many limitations. First, for the lack of considering multi-scale information embedded in different layers of AE, only deep features can be captured which may lose plenty of useful features. Second, when it comes to graph-based clustering, DSC could only focus on the relationship of node attributes while neglecting the structure of data, which inspires us to employ graph auto-encoders instead of AE for learning a better latent node representation.

To handle the aforementioned problem on graph-based clustering, many researches have made some attempts, of which can be simply fall into three categories: node attributes based, graph structure based, and both node attributes and graph structure based approach. The typical representative algorithm of node attributes based approach is K-means [37] which only considering the node attributes. Spectral [23], DeepWalk [38], GraEnc [1] and DNGR [39] are the most successful methods in the category of graph structure based approach, which only exploit the graph structure. The limitations of these approaches is obvious that they can only capture part of the graph information, which cannot usefully exploit the graph structure or the relationship between the node attributes and the graph structures.

On account of the earlier observation, the method based on graph auto-encoder approach have been widely suggested, which aims at capturing the node attribute and structural information behind the data. For instance, GAE [18] uses graph convolutional networks (GCN) as the encoder and reconstruct the graph structure in the encoder and VGAE [18] is the variational version of GAE. MGAE [21] is marginalized graph auto-encoder for graph-based clustering and it advances the auto-encoder to the graph domain. ARGAE [20] is adversarially regularized graph auto-encoder for graph embedding since regularization method is introduced to enable dimensionless data to learn and process the characteristics of data and ARVGAE [20] is the variational version of ARGAE. DAEGC [40] develops a variant of the graph attention network (GAT) [41] as the encoder part and jointly optimizes the embedding learning and graph-based clustering. GATE [22] is proposed for reconstructing both node attributes and the graph structure on graph-based datasets with self-attention mechanism.

Although graph auto-encoders based methods [18,20,20,40,40,22,42] have shown very effective in integrating

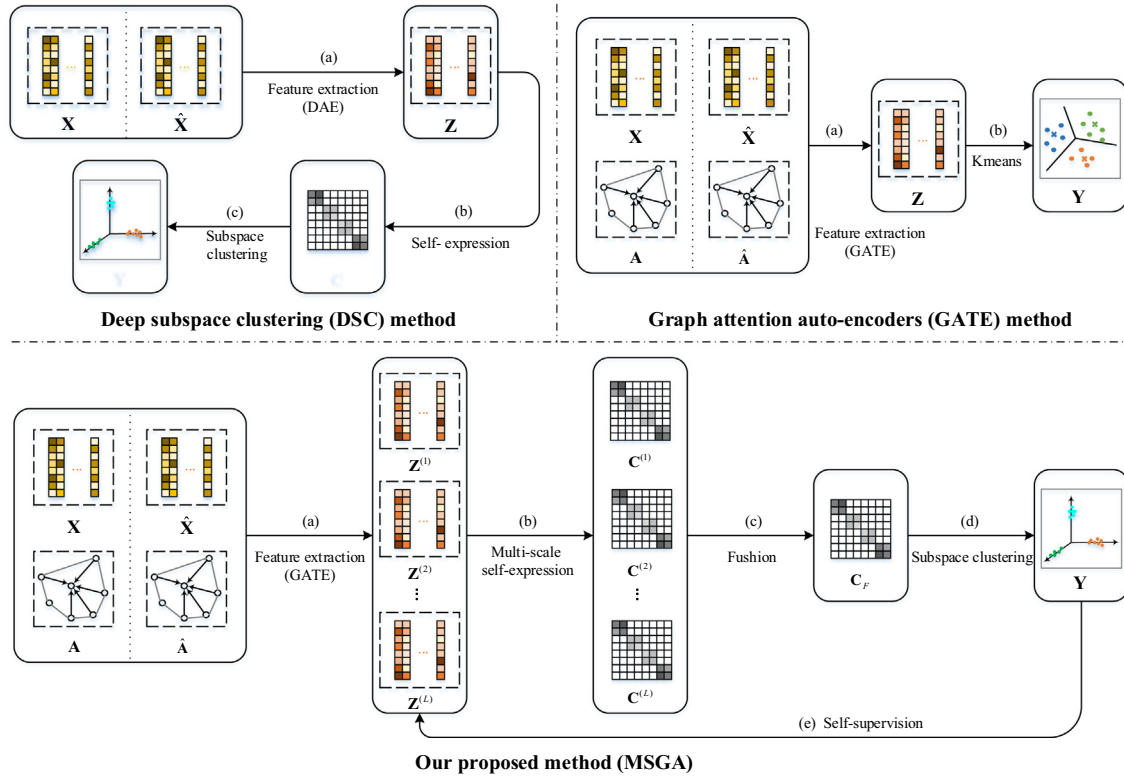


Fig. 1. Motivation of our proposed method. $\mathbf{X}$ and $\mathbf{A}$ are the node attribute matrix and the adjacency matrix, respectively. $\hat{\mathbf{X}}$ and $\hat{\mathbf{A}}$ are the reconstructed matrix. $\mathbf{Z}$ is the latent node representation matrix and $\mathbf{C}$ is the self-expression matrix. In our method, we fully exploit the multi-scale information embedded in different layers of encoder and fuse them as a better self-expression matrix $\mathbf{C}_f$. Moreover, we utilize the current clustering label $\mathbf{Y}$ to guide the learning of node representation $\mathbf{Z}$. We jointly optimize them in a unified framework so that the node representation can learn the clustering information.

graph structure information and node attributes, however, existing methods still have some issues need to be addressed. Many of them merely use a shallow model extracting features, which may be unable to capture the complex latent structure of large-scale datasets, while others use deep model but still neglect to fully utilize the feature information embedded in different layers. In addition, some methods do not employ a clustering-oriented loss function and some fail to utilize the clustering label guiding node representation learning effectively. All these problems lead to the suboptimal clustering result. In order to address the issues mentioned above, in this paper, we proposed our model called MSGA.

3. Our proposed model: MSGA

In this section, we introduce our proposed Multi-Scale Graph Attention Subspace Clustering Network (MSGA), where the overall architecture is shown in Fig. 2. Specifically, our proposed network consists of four different modules, i.e., feature extraction module, multi-scale self-expression module, self-supervised module and deep subspace clustering module. We jointly train and optimize all the module in a unified framework so that they can benefit from each others and help learn a latent node representation contained clustering information. We introduce each module in the following subsections.

3.1. Notations

For clearly illustrating our model, we first introduce the notations used in this paper. We use bold upper case letters for matrix, e.g., $\mathbf{A}$ and bold lower case letters for vectors, e.g., $\mathbf{a}$.

We analyze the clustering task based on the attributed graph which can be represented as $\mathbf{G} = (\mathbf{V}, \mathbf{E}, \mathbf{X})$. $\mathbf{V} = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_i, \dots, \mathbf{v}_n\}$ is the node set and $\mathbf{v}_i$ is the i^{th} node. $\mathbf{E} = \{\mathbf{e}_{ij}\}$ represents the edges between nodes since the topological structure of the attributed graph can be indicated by the adjacency matrix $\mathbf{A}$ ($\mathbf{A} \in \mathbb{R}^{n \times n}$). $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_i, \dots, \mathbf{x}_n\}$ ($\mathbf{X} \in \mathbb{R}^{n \times d_0}$) is the node attribute set where n denotes the number of nodes in the graph structured datasets, $\mathbf{x}_i \in \mathbb{R}^{d_0}$ is the feature vector of node i and d_0 is the dimension of the node attribute. $\mathbf{H}^{(i)} = \{\mathbf{h}_1^{(i)}, \mathbf{h}_2^{(i)}, \dots, \mathbf{h}_j^{(i)}, \dots, \mathbf{h}_n^{(i)}\}$ ($\mathbf{H}^{(i)} \in \mathbb{R}^{n \times d_i}$) is the latent node representation in the i^{th} layer of the encoder, where $\mathbf{h}_j^{(i)} \in \mathbb{R}^{d_i}$ is the latent node representation of the j^{th} node, d_i is the dimension of latent node representation in the i^{th} layer.

3.2. Feature extraction module

A basic component of our proposed MSGA is the feature extraction module, which is proposed for unsupervised learning on graph-structured data, using stacked encoder and decoder layers equipped with self-attention mechanisms to reconstruct node attributes and graph structure. In the feature extraction module, the graph structured data $(\mathbf{X}, \mathbf{A})$ is first input into the encoder, then the reconstructed sample $(\hat{\mathbf{X}}, \hat{\mathbf{A}})$ is received from the decoder after the nonlinear transformation.

For the encoder, the node attribute $\mathbf{x}$ is considered as the initial node representation $\mathbf{h}^{(1)}$ and the node representation of next layer $\mathbf{h}^{(i)}$ is generated by attending over their neighbors' representations. The target of it is to learn the latent node representations $\mathbf{H}^{(i)}$. To

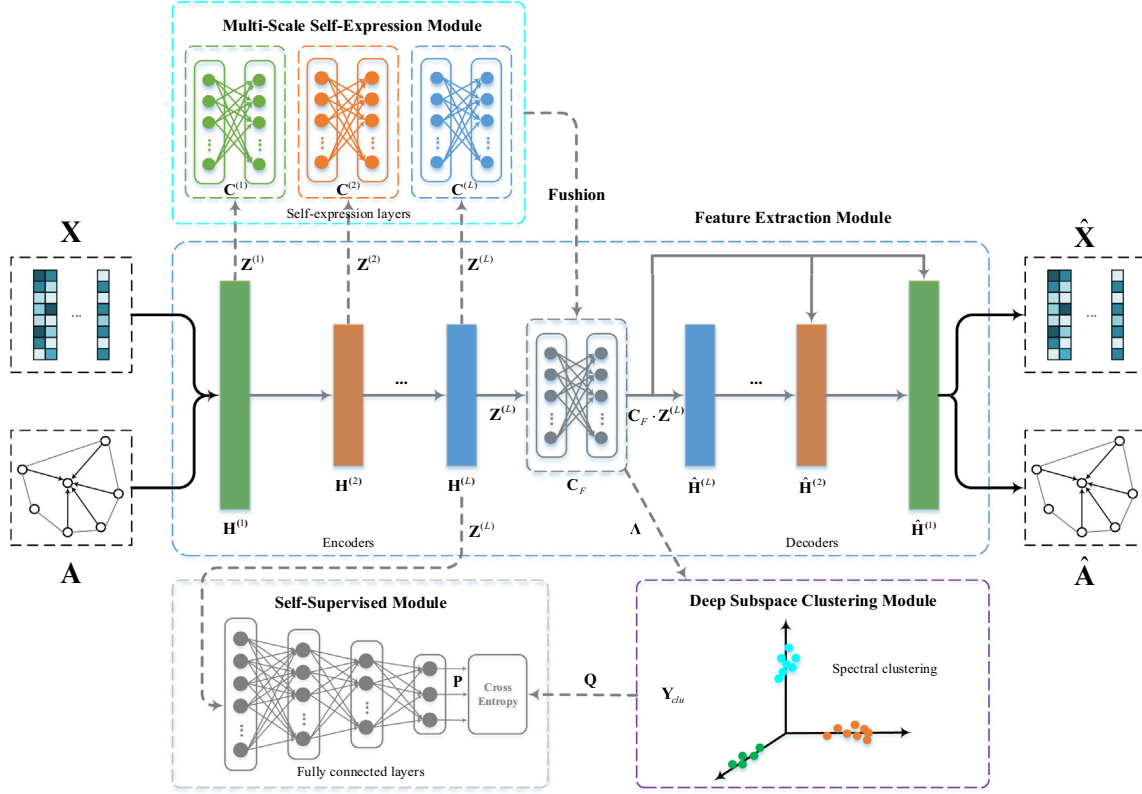


Fig. 2. Architecture of the proposed method. Our network contains four modules: a) feature extraction module, which employs graph attention auto-encoders to extract node representations from the node attribute and graph structure, b) multi-scale self-expression module, which learns the self-expression coefficient matrix of different layers in the encoder and fuses them together, c) self-supervised module, which guides the learning of the node representation via building a self-supervision path back to the encoder, d) deep subspace clustering module, spectral clustering is applied and the clustering labels are provided back to the self-supervised module.

ease of presentation, we introduce it with the simplest case, i.e., the i^{th} hidden-layer of the graph attention encoder as follows:

$$\mathbf{H}^{(i)} = g(\mathbf{X}, \mathbf{A} | \Theta^{(i)}), \quad (1)$$

where $\Theta^{(i)} = \{\mathbf{W}^{(i)}, \mathbf{v}_s^{(i)}, \mathbf{v}_r^{(i)}\}$ is a parametric network with the trainable parameters $\mathbf{W}^{(i)} \in \mathbb{R}^{d_i \times d_{i-1}}$, $\mathbf{v}_s^{(i)} \in \mathbb{R}^d$ and $\mathbf{v}_r^{(i)} \in \mathbb{R}^d$. $g(\cdot)$ denotes the nonlinear activation function adopted in [22] and $\mathbf{Z}^{(i)} = \mathbf{H}^{(i)}$ in the encoder part.

For the decoder, the encoding process has been reversed to reconstruct node attributes. Assume that the encoder has a total L layer, $\mathbf{CZ}^{(L)}$ is feed into the decoder network for reconstructing the graph structured data $(\hat{\mathbf{X}}, \hat{\mathbf{A}})$, where $\hat{\mathbf{H}}^{(L)} = \mathbf{CZ}^{(L)}$, $\mathbf{C}$ is the self-expression matrix (see Section 3.3) and $\mathbf{Z}^{(L)}$ is the latent node representation on L^{th} layer of the encoder. To ease of presentation, we introduce the process of the node representation reconstructing in the graph attention decoder as follows:

$$\hat{\mathbf{H}}^{(i)} = \hat{g}(\mathbf{CZ}^{(L)}, \mathbf{A}^{(L)} | \hat{\Theta}^{(i)}), \quad (2)$$

where $\hat{\mathbf{H}}^{(i)} = \{\hat{\mathbf{h}}_1^{(i)}, \hat{\mathbf{h}}_2^{(i)}, \dots, \hat{\mathbf{h}}_j^{(i)}, \dots, \hat{\mathbf{h}}_n^{(i)}\}$ ($\hat{\mathbf{H}}^{(i)} \in \mathbb{R}^{n \times d_i}$) is the latent node representation in the i^{th} layer of the decoder, $\hat{\Theta}^{(i)} = \{\hat{\mathbf{W}}^{(i)}, \hat{\mathbf{v}}_s^{(i)}, \hat{\mathbf{v}}_r^{(i)}\}$ is a parametric network with the trainable parameters $\hat{\mathbf{W}}^{(i)} \in \mathbb{R}^{d_i \times d_{i-1}}$, $\hat{\mathbf{v}}_s^{(i)} \in \mathbb{R}^d$ and $\hat{\mathbf{v}}_r^{(i)} \in \mathbb{R}^d$. $\hat{g}(\cdot)$ denotes the nonlinear activation function adopted in [22].

The reconstructed node attribute (i.e. $\hat{\mathbf{x}}$) is considered as the node representations of the last layer in the decoder (i.e. $\hat{\mathbf{h}}^{(1)}$). We can use residual sum of squares to minimize the reconstruction loss of node attribute which can be calculated as:

$$\mathcal{L}_{att} = \min_{\Theta} \frac{1}{2} \cdot \sum_{i=1}^n \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_F^2. \quad (3)$$

The reconstruction loss of the graph structure should be minimized by the following equation:

$$\mathcal{L}_{gra} = \min_{\Theta} - \sum_{i=1}^n \sum_{j \in \mathcal{N}_i} \log \left(\frac{1}{1 + \exp(-\mathbf{h}_i^T \mathbf{h}_j)} \right). \quad (4)$$

Hence the total loss of the feature extraction module can be formulated as the follows:

$$\mathcal{L}_1 = \mathcal{L}_{att} + \mathcal{L}_{gra}. \quad (5)$$

3.3. Multi-scale self-expressive module

Excellent subspace clustering methods rely on the self-expression property of data, which means that each data point in a union of subspace can be expressed as a linear combination of other data. In this paper, our multi-scale self-expressive module propose a novel approach of obtaining the final self-expression coefficient matrix through integrating each single self-expression coefficient from all the latent node feature matrix $\mathbf{H}^{(i)}$.

As shown in deep subspace clustering network (DSC) [17], the self-expression layer is a fully-connected layer with out any activation and bias, which has proven effective in conventional subspace clustering. The weights of the self-expressive layer correspond to the coefficient matrix $\mathbf{C}$, which is further used to construct an affinity matrix Λ for spectral clustering. The original object function in

DSC consists of regularization and self-expression which can be summarized as follows:

$$\min_{\mathbf{C}} \|\mathbf{C}\|_p + \frac{1}{2} \|\mathbf{Z} - \mathbf{C}\mathbf{Z}\|_F^2 \quad \text{s.t.} \quad (\text{diag}(\mathbf{C}) = 0), \quad (6)$$

where $\mathbf{C} \in \mathbb{R}^{n \times n}$ is the self-expression coefficient matrix, $\mathbf{Z} \in \mathbb{R}^{n \times d}$ is the output feature matrix of the encoder, $\|\cdot\|_p$ represents an arbitrary regularization norm, such as the l_1 norm.

Since DSC neglects the multi-scale information indwelled in other layers of the encoder, to this end, we develop the original self-expression method by considering the multi-scale information from different layers. For clarity, assume that the input graph structured data $(\mathbf{X}, \mathbf{A})$ after the i^{th} layer encoded as $\mathbf{Z}^{(i)} \in \mathbb{R}^{n \times d_i}$ ($i = 1, 2, \dots, L$), where d_i denotes the size of the hidden dimension in i^{th} layer and L denotes the total layer of the encoder. Notice that, each layer of the encoder can calculate the self-expression coefficient matrix $\mathbf{C}^{(i)}$. For instance, $\mathbf{C}^{(1)}$ represents the self-expression matrix of 1^{st} layer in the encoder, which can be constrained by the follows:

$$\min_{\mathbf{C}^{(1)}} \frac{1}{2} \|\mathbf{Z}^{(1)} - \mathbf{C}^{(1)}\mathbf{Z}^{(1)}\|_F^2. \quad (7)$$

To obtain a fused multi-scale self-expression matrix $\mathbf{C}_F$ which is used to calculate the affinity matrix for spectral clustering, it is of crucial importance to effectively fuse the self-expression matrix $\mathbf{C}^{(i)}$ in different layers together since bad fusing method would even make the clustering performance worse. We fuse them through a adaptive method which can be formulated as follows:

$$\mathbf{C}_F = \frac{\sum_{i=1}^L \tau_i \cdot \mathbf{C}^{(i)}}{\sum_{i=1}^L \tau_i}, \quad (8)$$

where τ_i is a trainable variable which help balance the weight of each self-expression matrix, L denotes the layer of the encoder.

As aforementioned, we can completely use the information embedded in each layer via integrating different self-expression matrix, and the multi-scale self-expression loss can be formulated as follows:

$$\mathcal{L}_{mss} = \min_{\mathbf{C}^{(i)}} \frac{1}{2L} \cdot \sum_{i=1}^L \|\mathbf{Z}^{(i)} - \mathbf{C}^{(i)}\mathbf{Z}^{(i)}\|_F^2 \quad \text{s.t.} \quad (\text{diag}(\mathbf{C}^{(i)}) = 0). \quad (9)$$

Besides, in order to prevent the self-expression coefficient matrix $\mathbf{C}^{(i)}$ of each layers become too spare, we introduce the regularization loss which can be formulated as follows:

$$\mathcal{L}_{reg} = \min_{\mathbf{C}^{(i)}} \frac{1}{L} \cdot \sum_{i=1}^L \|\mathbf{C}^{(i)}\|_p \quad \text{s.t.} \quad (\text{diag}(\mathbf{C}^{(i)}) = 0), \quad (10)$$

Hence, the total loss of the multi-scale self-expression module can be formulated as follows:

$$\mathcal{L}_2 = \mathcal{L}_{mss} + \mathcal{L}_{reg}. \quad (11)$$

3.4. Deep subspace clustering module

As mentioned in multi-scale self-expressive module, once the final self-expression matrix $\mathbf{C}_F$ is obtained, the deep subspace clustering module can use it to construct an affinity matrix Λ for spectral clustering [23], which can be formulated as follows:

$$\Lambda = \frac{1}{2} (|\mathbf{C}_F| + |\mathbf{C}_F^T|). \quad (12)$$

Then, we can obtain the clustering result $\mathbf{Y}_{clu}$ using a spectral method on the affinity matrix Λ .

3.5. Self-supervised module

The self-supervised module is employed by using a fully connected layer for classification and calculating the cross-entropy loss between the classification result $\mathbf{P}$ and the clustering result $\mathbf{Q}$.

The motivation of this module is using the current clustering label self-supervise the learning of node representation $\mathbf{H}^{(i)}$ and making the node representation contain current clustering information. Though that the result of current spectral clustering might not represent the correct class label of all the samples, still remains many useful information about the data. Once the better node representation is achieved, the clustering performance would be improved and a more accurate clustering label may be obtained for self-supervising next epoch.

Since the result of spectral clustering is an n dimensional vector which represents the relationship into n subspaces (i.e., clusters). To this end, we design the dimension of the fully connected layers as $\{d_L \times D_1 \times D_2 \times D_3 \times k\}$, where d_L is the dimension of the extracted node representation $\mathbf{Z}^{(L)}$, and D_1, D_2 and D_3 are the dimension of the fully connected layers respectively. Denoting $\mathbf{P}$ as the pseudo label which is the classification label of the fully connected layers, where $\mathbf{P} \in \mathbb{R}^{n \times k}$, n is the number of nodes and k is the number of clusters. Treating $\mathbf{Y}_{clu} \in \mathbb{R}^{n \times k}$, which is the clustering label of the deep clustering module as the learning target of $\mathbf{P}$, we can use cross-entropy loss to constrain this module, which is formulated as follows:

$$\min_{\mathbf{P}} - \sum_{i=1}^n \sum_{j=1}^k \mathbf{P}(i, j) \log \mathbf{Y}_{clu}(i, j), \quad (13)$$

where $\mathbf{Y}_{clu}(i, j)$ is the j^{th} label of the i^{th} node of the spectral clustering label $\mathbf{Y}_{clu}$, and $\mathbf{P}(i, j)$ is the j^{th} label of the i^{th} node of the classification label $\mathbf{P}$.

Notice that the clustering label $\mathbf{Y}_{clu}$ changes each epoch along with training process, it would be hazardous to update $\mathbf{P}$ with $\mathbf{Y}_{clu}$ since it may obstruct learning and convergence of the model. To tackle this issue, we adopt a training trick by setting a iteration T_0 and a temporary label $\mathbf{Q} \in \mathbb{R}^{n \times k}$. We use $\mathbf{Q}$ instead of $\mathbf{Y}_{clu}$ as the learning target for the self-supervision in Eq. (14). We consider $\mathbf{Y}_{clu}$ as the initial value of $\mathbf{Q}$ and update $\mathbf{Q}$ with $\mathbf{Y}_{clu}$ every T_0 epoches, so that the learning target $\mathbf{Q}$ is stable in T_0 epoches. The loss function of this module can be modified as follows:

$$\mathcal{L}_3 = \mathcal{L}_{sup} = \min_{\mathbf{P}} - \sum_{i=1}^n \sum_{j=1}^k \mathbf{P}(i, j) \log \mathbf{Q}(i, j). \quad (14)$$

3.6. Training multi-scale graph attention subspace clustering network

To obtain the end-to-end training work, we integrate Eq. (5), Eq. (11) and Eq. (14) together as the total loss function of MSGA which can be formulated as follows:

$$\mathcal{L}_{total} = \min_{(\Theta, \mathbf{C}, \mathbf{P}, \mathbf{Z})} \mathcal{L}_{att} + \lambda_1 \cdot \mathcal{L}_{gra} + \lambda_2 \cdot \mathcal{L}_{reg} + \lambda_3 \cdot \mathcal{L}_{mss} + \lambda_4 \cdot \mathcal{L}_{sup}, \quad (15)$$

where $\lambda_1, \lambda_2, \lambda_3$ and λ_4 are the four tradeoff parameters for the above loss function.

Algorithm 1. MSGA for subspace clustering

Input: Input the graph-structured dataset.
Parameter: Tradeoff parameters: $\lambda_i (i = 1, 2, 3, 4)$, number of clusters: k , maximum iteration: T , T_0 , $iter = t = 1$ and learning rate: lr .
Output: The best clustering label $\mathbf{Y}_{clu}$.
1: Initialize the weight of the multi-scale self-expression layer with constant 10^{-4} , letting $\tau_i = 1, (i = 1, 2, 3, \dots, L)$.
2: **while** $iter < T$ **do**
3: Obtain $\mathbf{Z}^{(i)}, (i = 1, 2, 3, \dots, L)$ by Eq. (1)
4: Obtain $\mathbf{C}_F$ by Eq. (8)
5: Obtain Λ by Eq. (12)
6: Obtain $\mathbf{Y}_{clu}$ by applying spectral clustering on Λ
7: Obtain $\mathbf{P}$ by feeding $\mathbf{Z}^{(L)}$ into fully connected layers
8: Let $\mathbf{Q} \leftarrow \mathbf{Y}_{clu}$
9: **while** $t < T_0$ **do**
10: Update $\mathbf{P}, \mathbf{C}^{(i)}, \Theta^{(i)}$ and $\tau_i, (i = 1, 2, \dots, L)$ by Eq. (15)
11: Let $t \leftarrow t + 1$
12: **return** clustering label $\mathbf{Y}_{clu}$
13: **end while**
14: Let $iter \leftarrow iter + 1$
15: **end while**

Our method is summarized in Algorithm 1 and the training process on MSGA can be concluded as follows. First, the node attribute matrix $\mathbf{X}$ and the adjacency matrix $\mathbf{A}$ should be feed into the feature extraction module and the latent node representation matrix $\mathbf{H}^{(i)}$ (i.e. $\mathbf{Z}^{(i)} (i = 1, 2, 3, \dots, L)$) from the feature extraction module is obtained via minimizing the node attribute loss $\mathcal{L}_{att}$ (i.e. Eq. (3)) and graph structure loss $\mathcal{L}_{gra}$ (i.e. Eq. (4)). Then, in the multi-scale self-expression module, individual multi-scale self-expression coefficient matrix $\mathbf{C}^{(i)} (i = 1, 2, 3, \dots, L)$ is learned from the corresponding layers of the encoder by minimizing the multi-scale self-expression loss $\mathcal{L}_{mss}$ (i.e. Eq. (9)) and the regularization loss $\mathcal{L}_{reg}$ (i.e. Eq. (10)). Meanwhile, the final fused self-expression coefficient matrix $\mathbf{C}_F$ would be calculated by (i.e. Eq. (8)) with the adaptive method integrating the multi-scale self-expression coefficient matrix. After that, the affinity matrix Λ constructed by Eq. (12) is obtained and the clustering label $\mathbf{Y}_{clu}$ would be calculated in the deep clustering module. Finally, the temporary clustering label $\mathbf{Q}$ would be obtained from $\mathbf{Y}_{clu}$ and set as the target of self-supervision for the learning of node representation $\mathbf{H}^{(i)}$ via minimizing the self-supervised loss $\mathcal{L}_{sup}$ (i.e. Eq. (14)) on $\mathbf{P}$ and $\mathbf{Q}$ in the self-supervised module, since then, all the training parameters (i.e. $\Theta, \mathbf{C}, \mathbf{P}, \mathbf{Z}$) will be updated.

We use $\mathcal{L}_{total}$ (i.e. Eq. (15)) to jointly train and optimize all the module in one-step so that the embedding and clustering approach are integrated in a unified work. They can benefit from each other and helping to learn a better node representation containing clustering information, which would greatly improve the clustering performance.

4. Experimental analysis

In this section, we conduct some significant architectures on graph structured datasets and real-world datasets to evaluate the performance our MSGA. Specially, we provide the following access which contains the code of MSGA on HHAR and USPS. <https://github.com/wangtong627/Multi-Scale-Graph-Attention-Subspace-Clustering-Network>.

4.1. Experimental setting

Graph datasets. The performance of our MSGA is evaluated on three graph datasets and two large-scale real-world datasets. The statistics are shown in Table 1 and the detailed descriptions are the followings:

- **Cora** [43] is a machine learning papers dataset which contains 2,708 nodes machine learning papers divided into seven classes and 5,429 links between them. Each node is represented by a binary word vector of 1433 dimensions.
- **Citeseer** [43] is a scientific papers dataset which contains 3,312 nodes from six classes and 4,732 links between them. Same as Cora, the links are citation relationships between the documents and each node is described by a binary word vector of 3,703 dimensions.
- **Wiki** [44] is a webpage network dataset which contains 2,405 nodes from 19 classes and 17,981 links between them. Each node of Wiki is a webpage represented by a tf-idf weighted word vector with 4,973 dimensions.

Real-world datasets. To capture the structural information, we employ K nearest neighbor (KNN) method to construct an undirected K-nearest neighbor graph. For each node (i.e. $\mathbf{x}$), we calculate its top-K similar neighbors and connect them with edges. In this way, we can get the adjacency matrix $\mathbf{A}$ from the non-graph real-world dataset. Specially, for different real-world datasets, we set $K = 3$ in USPS and $K = 5$ in HHAR according to [45]. The statistics are shown in Table 2 and the detailed descriptions are the followings:

- **USPS** [45] contains 9298 gray-scale handwritten digit images and the size of each image is 16×16 pixels. The features of each image is a vector of 256 dimensions represents the gray value of pixels pointing to images.
- **HHAR** [45] contains 10299 smart phones and small watches sensor recorders and all the recorders are divided into 6 categories of human activities. The features of each recorder is a vector of 561 dimensions.

Metrics. In order to evaluate the effectiveness of our proposed approach, three representative and influential metrics are used in this experiment. Denote $\mathbf{y} = \{\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n\}$ is the ground-truth label, $\mathbf{c} = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_n\}$ is the predicted cluster label.

- **Unsupervised Clustering Accuracy (ACC)** [46] is the first metric which is formulated as follows:

$$ACC = \max_m \frac{\sum_{i=1}^n 1\{\mathbf{y}_i = m(\mathbf{c}_i)\}}{n}, \quad (16)$$

where m is a mapping function which ranges over all possible one-to-one mappings between the true label $\mathbf{y}$ and predicted cluster label $\mathbf{c}$. The optimal mapping function can be efficiently computed by Hungarian algorithm [47].

- **Normalized Mutual Information (NMI)** [48] is the second which is used to measure the normalized similarity of the true label $\mathbf{y}$ and the corresponding predicted cluster label $\mathbf{c}$ and can be represent as follows:

$$NMI = \frac{I(\mathbf{y}, \mathbf{c})}{\max[H(\mathbf{y}), H(\mathbf{c})]}, \quad (17)$$

where I is the mutual information between the true label $\mathbf{y}$ and predicted cluster label $\mathbf{c}$, $H(\mathbf{y})$ and $H(\mathbf{c})$ is the entropy of the true label $\mathbf{y}$ and predicted cluster label $\mathbf{c}$.

Table 1

The statics of the graph datasets.

Dataset	Nodes	Edges	Features	Classes
Cora [43]	2708	5429	1433	7
Citeseer [43]	3327	4732	3703	6
Wiki [44]	2405	17981	4973	19

Table 2

The statics of the real-world datasets.

Dataset	Type	Nodes	Features	Classes
USPS [45]	Image	9298	256	10
HHAR [45]	Record	10299	561	6

- **Adjusted Rand Index (ARI)** [49] is the third metric which is a effective method for comparing two partitions and can be formulated as follows:

$$ARI = \frac{RI - E[RI]}{1 - E[RI]}, \quad (18)$$

where $RI = \frac{A+B}{C}$, $E[RI]$ is the expected value of RI when the partitions are made at random, C_2^N is all the possible combination pairs of samples, A represents the number of sample combination pairs in the same partition both $\mathbf{y}$ and $\mathbf{c}$, B represents the number of sample combination pairs not in both $\mathbf{y}$ and $\mathbf{c}$.

To better illustrate the effectiveness of the method, each term performed 10 times and mark the average results avoiding some extreme cases. In addition, for all the metrics above, a higher value represents a better performance.

Implementation details. Our MSGA is implemented by TensorFlow [50] and optimized by ADAM [51]. In order to completely evaluate the performance of our proposed MSGA, we train the network with 100 epochs and the learning rate is 10^{-5} . Making our method more comparable, we set the dimension of the feature extraction module the same as $\{d_0 \times 512 \times 512 \times 512 \times 512 \times d_0\}$, where d_0 is the dimension of the node attribute. In the Self-Supervised Module, we employ ReLU as the non-linear activation function and the dimension of the fully connected layers is set as $\{d_L \times 128 \times 64 \times 32 \times k\}$, where d_L is the dimension of the output node representation (i.e. $\mathbf{Z}^{(L)}$) and k is the number of clusters. Moreover, we set $T_0 = 5$ which means $\mathbf{Q}$ would be updated by $\mathbf{Y}_{clu}$ every 5 epoches. For analyzing the effect of dif-

ferent tradeoff parameters for the performance, we adjust $\lambda_1, \lambda_2, \lambda_3$ and λ_4 from 10^{-2} to 10^2 .

Comparison algorithms. To illustrate the performance of our methods, we compared our proposed MSGA with several state-of-the-art clustering algorithms, which can be summarized as follows.

- **Using node attributes.**

K-means [37] is a basic clustering method to get k clusters through repeated iteration.

- **Using graph structures.**

Spectral [23] is a clustering method based on graph theory, which treats data as points connected by edges in space.

DeepWalk [38] is a method of network embedding, which learns a social representation of a network by truncated random walk.

GraEnc [1] is an auto-encoder based graph-based cluster model. The key component is a deep neural network with sparse auto-encoder as its building block.

DNGR [39] is deep neural networks for learning graph representations, a method for low dimensional vectorization representation of graph structure.

- **Using both node attributes and graph structures.**

VGAE [18] uses GCN as the encoder and reconstruct the graph structure in the encoder.

ARGAE [20] is adversarially regularized graph auto-encoder for graph embedding. Regularization method is introduced to enable dimensionless data to learn and process the characteristics of data.

ARVGAE [40] is the variational version of ARGAE.

MGAE [21] is marginalized graph auto-encoder for graph-based clustering. It advances the auto-encoder to the graph domain.

GATE [22] is a neural network architecture for unsupervised representation learning on graph-structured data with self-attention mechanism.

DAEGC [40] is a graph-attention based auto-encoder which jointly learns and optimizes the embedding representations for graph-based clustering.

SDCN [45] integrates structural information into deep clustering by combining the representation of auto-encoder and GCN.

4.2. Experimental results

The clustering results of Cora, Citeseer and Wiki are reported in Table 3 and the results of HHAR and USPS are reported in Table 4.

Table 3

Clustering results of various methods on three graph datasets

	Cora			Citeseer			Wiki		
	ACC	NMI	ARI	ACC	NMI	ARI	ACC	NMI	ARI
Kmeans [37]	0.492	0.321	0.230	0.540	0.305	0.278	0.417	0.440	0.150
Spectral [23]	0.367	0.127	0.031	0.239	0.056	0.010	0.220	0.182	0.015
DeepWalk [38]	0.484	0.327	0.243	0.337	0.088	0.092	0.385	0.34	0.170
GraEnc [1]	0.325	0.109	0.005	0.225	0.033	0.010	0.207	0.121	0.005
DNGR [39]	0.419	0.318	0.142	0.356	0.180	0.043	0.376	0.359	0.180
VGAE [18]	0.502	0.329	0.254	0.467	0.260	0.205	0.450	0.167	0.263
ARGAE [20]	0.640	0.449	0.352	0.573	0.350	0.341	0.380	0.344	0.112
ARVGAE [20]	0.638	0.450	0.374	0.544	0.261	0.245	0.386	0.338	0.107
MGAE [21]	0.684	0.511	0.444	0.660	0.412	0.413	0.514	0.485	0.350
GATE [22]	0.658	0.527	0.451	0.616	0.401	0.381	0.465	0.428	0.316
DAEGC [40]	0.704	0.528	0.496	0.672	0.397	0.410	0.478	0.449	0.324
SDCN [45]	0.712	0.535	0.506	0.659	0.387	0.401	0.498	0.458	0.346
MSGA	0.747	0.578	0.519	0.698	0.433	0.415	0.522	0.481	0.363

Compared with other state-of-the-art methods, we have the following observations:

- Our proposed MSGA consistently outperforms all the compared methods on all the datasets. In particular, it can be calculated that the accuracy of MSGA improves by a margin of over 5% on Cora and Citeseer and over 2% than the suboptimal method. This can be concluded that our proposed method can extract embedded information from the multi-scale module and learn a more discriminative self-expression coefficient matrix helping map into a more accurate subspace for clustering. Moreover, the self-supervised module helps to learn a robust latent node representations, thereby improving the clustering performance.
- Among the method in the third category, MGAE performs better than VGAE, ARGAE and ARVGAE, which indicates that consider to reconstruct node attributes and graph structure together for the node representations can have better performance. By contrast, our MSGA reconstructs the representation of both node attributes (with GCN method) and graph structures (with attention mechanism) in the feature extraction module.
- DAEGC and GATE show superior performance especially on Cora dataset, which indicates that employ attention mechanism can make beneficial contribution to clustering. Our MSGA employ attention mechanism to reconstruct the graph structure in the feature extraction module.

Table 4

Clustering results of various methods on two real-world datasets.

	HHAR			USPS		
	ACC	NMI	ARI	ACC	NMI	ARI
Kmeans [37]	0.599	0.586	0.460	0.668	0.626	0.545
Spectral [23]	0.533	0.635	0.505	0.723	0.750	0.691
VGAE [18]	0.713	0.629	0.514	0.561	0.510	0.409
ARGAE [20]	0.729	0.643	0.528	0.668	0.641	0.576
MGAE [21]	0.751	0.687	0.653	0.724	0.701	0.652
GATE [22]	0.748	0.718	0.560	0.704	0.712	0.644
DAEGC [40]	0.765	0.691	0.603	0.735	0.711	0.633
SDCN [45]	0.842	0.799	0.728	0.780	0.795	0.718
MSGA	0.851	0.816	0.730	0.801	0.819	0.755

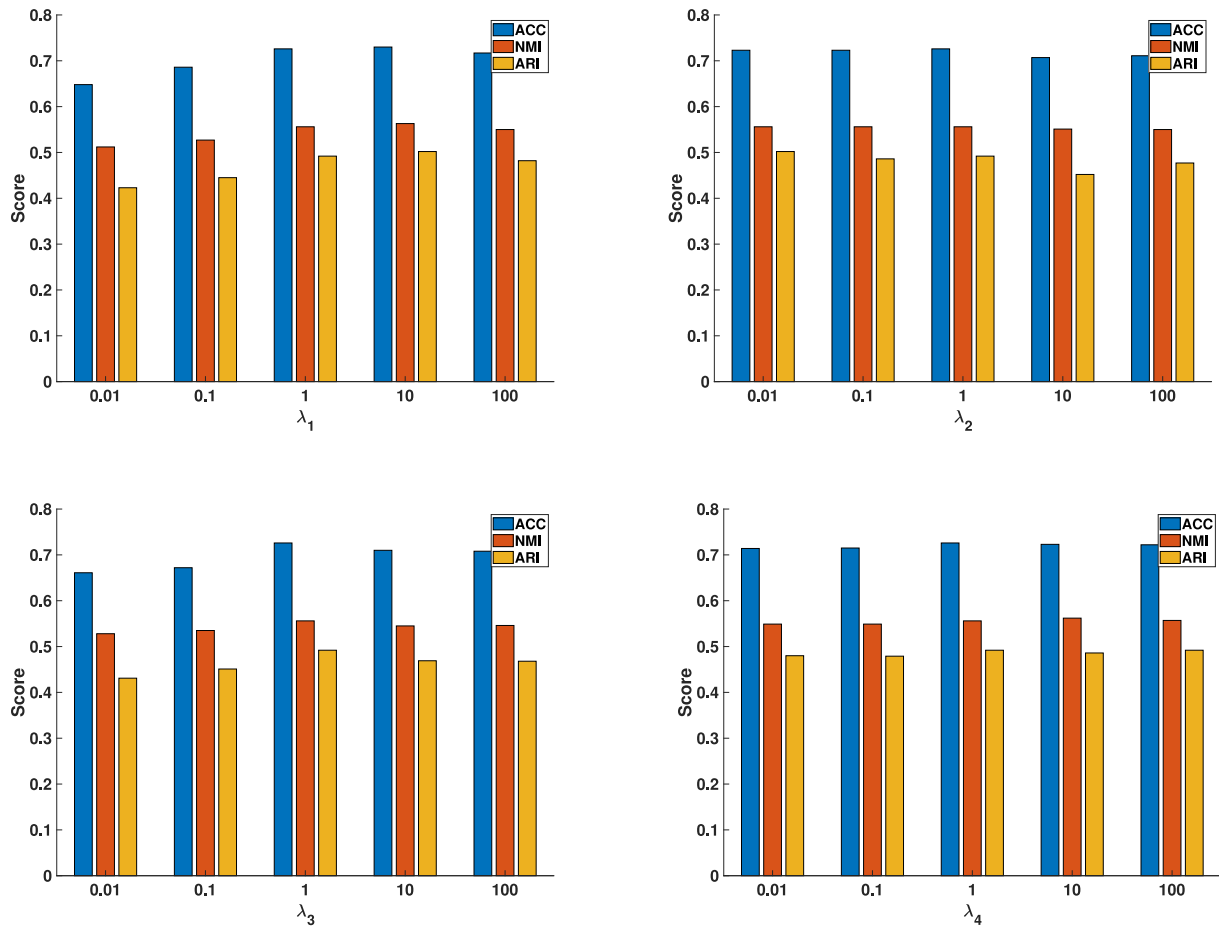


Fig. 3. Parameter analysis on λ_1 , λ_2 , λ_3 and λ_4 .

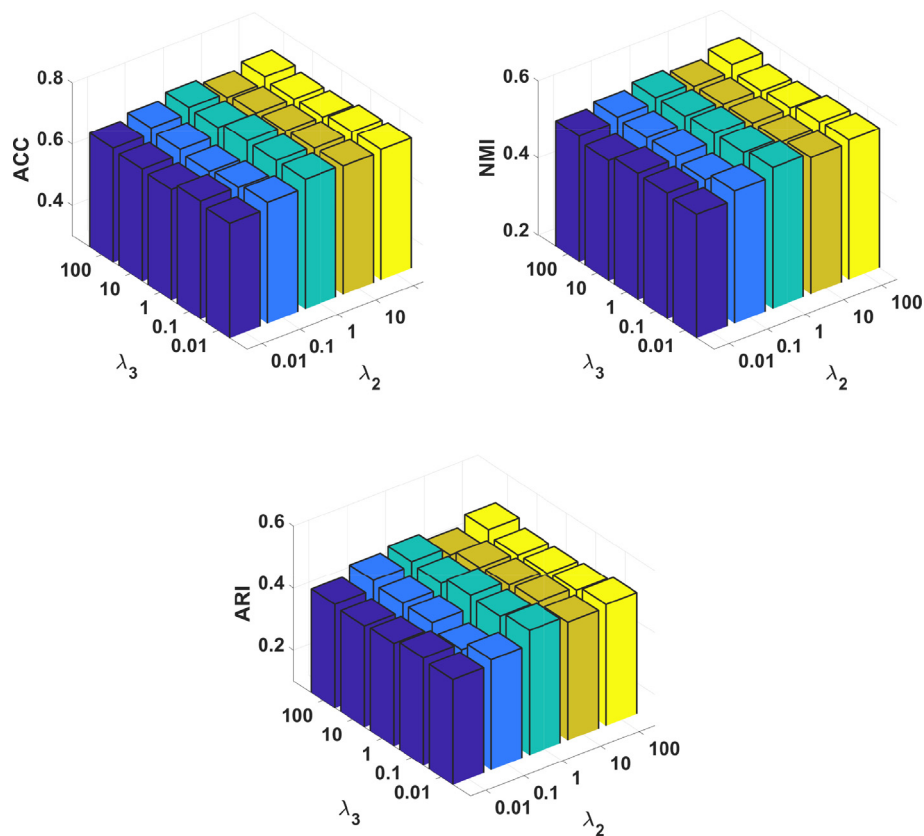


Fig. 4. Parameter analysis on λ_2 and λ_3 .

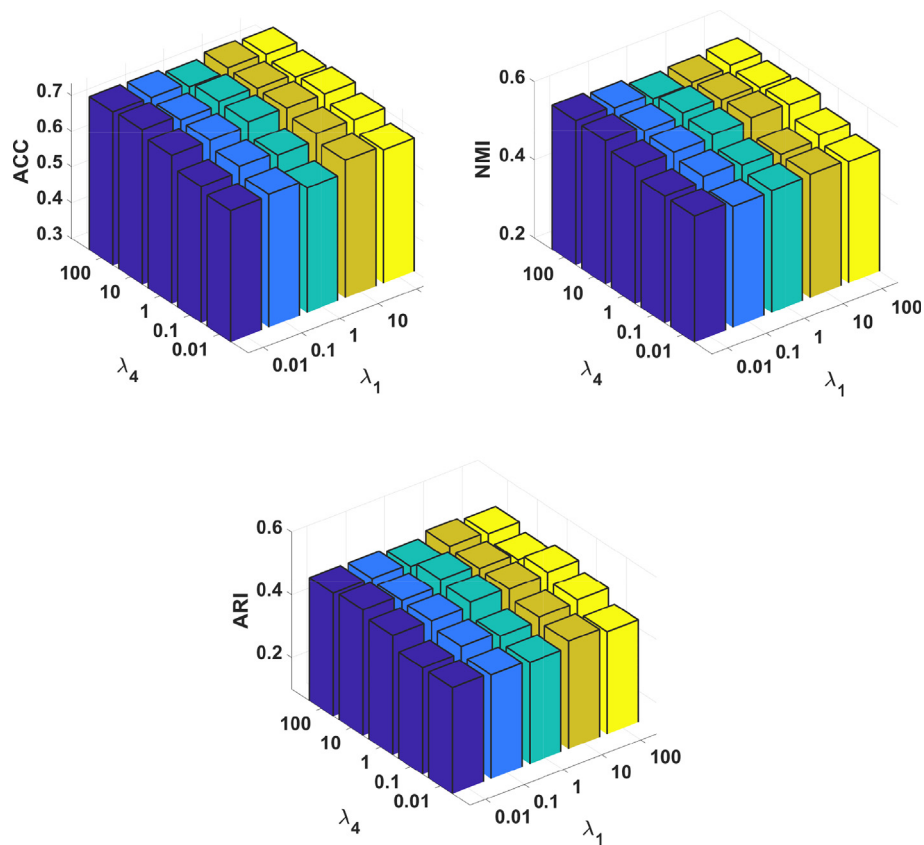


Fig. 5. Parameter analysis on λ_1 and λ_4 .

- DAEGC and SDCN outperforms other compared methods, which is because they train and optimize all the module in one-step, which makes different modules enhance each other. Our proposed MSGA employs the same training strategy and can effectively obtain the node representation with useful clustering information.
- As Kmeans only utilizes node attribute while spectral method only considers structure information, both of them lead to the suboptimal performance. Moreover, compared with these two shallow models, our MSGA is a deep model with hidden layers which has a superior node representation learning ability.
- Since SDCN utilizes the structure information, it achieves better results than most of the methods in the large-scale real-world datasets. However, all the methods except MSGA fail to consider the multi-scale information embedded in different layers. As a result, our proposed MSGA shows the superiority on both graph datasets and real-world datasets.

4.3. Parameters analysis

In this subsection, we employ controlling variables method to analyze the impact of four tradeoff parameters $\lambda_1, \lambda_2, \lambda_3$ and λ_4 on Cora dataset.

We analyze the impact of each parameter in MSGA via changing the specific parameter from 10^{-2} to 10^2 and other parameters are set as 1, respectively. The average performance is shown in Fig. 3, from which we can easily observe that λ_1 and λ_3 have relatively more effect on the model. Specifically, the performance turns with the same tendency as λ_1 , which indicates that using attention mechanism help reconstruct graph structure has a effective impact. Besides, the performance varies with λ_3 , which can be concluded that utilizing multi-scale information can improve the clustering performance.

In order to figure out the combined effect of the trade-off parameters, we attempt to adjust two of them at the same time seeing whether would have a significant change on the clustering performance. Since λ_2 and λ_3 are the parameter of regularization $\mathcal{L}_{reg}$ and multi-scale self-expression loss $\mathcal{L}_{mss}$ used in the same module (i.e. multi-scale self-expression module), we divide λ_2, λ_3 into a group and λ_1, λ_4 into an other for further analysis.

To be more intuitive, surface plots of Fig. 4 and Fig. 5 are presented to visualize their influences on evaluation metrics (ACC, NMI, ARI) at different values from 10^{-2} to 10^2 while other values fix as 1. In each figure, the x-axis and y-axis denote the weight of testing parameter while z-axis denote the score of the evaluation metric.

As shown in Fig. 4 and Fig. 5, the corresponding evaluation metrics ACC, NMI, ARI will largely remain stable, which means our model is still robust holistically on multi-scale approach. Specifically, compared with λ_3, λ_2 shows a more regular change to the clustering performance. The results improves when λ_2 changes from 0.01 to 1 and then starts to flatten out when λ_2 is over 1. The reason can be concluded that the attempt of reconstructing the graph structure can significantly improve the learning of node representation, and then the model will converge and the performance may become stable. We can also notice that even though sometimes the clustering performance may slightly fluctuate, the performance has the same trend of change as λ_4 in Fig. 5, which means the clustering result will be improved with appropriate self-supervised information and excessive self-supervision loss may make the result slightly fluctuate.

Despite the general performance changes slightly (as a result of robustness), the importance of parameters cannot be ignored because they are still important to the clustering task. From the present results and the analysis aforementioned, we can almost achieve the best clustering performance in terms of ACC, NMI and ARI by $\lambda_1 = 10, \lambda_2 = 0.01, \lambda_3 = 100$ and $\lambda_4 = 0.1$.

4.4. Convergence analysis

In this subsection, we illustrate the convergence curve of our MSGA on two graph dataset (i.e. Cora and Citeseer) and two real-world dataset (i.e. USPS and HHAR). The result is demonstrated in Fig. 6, which shows that MSGA achieves convergence after 40 training epochs on Cora and Citeseer and 60 epochs on USPS and HHAR.

At the beginning, the value of the loss function all decrease dramatically in different ranges and then tend to “flat”. Moreover, we use the clustering labels to guide the leaning of node representation in the self-supervised module. Since the constant change of the target would obstruct learning and convergence of the model,

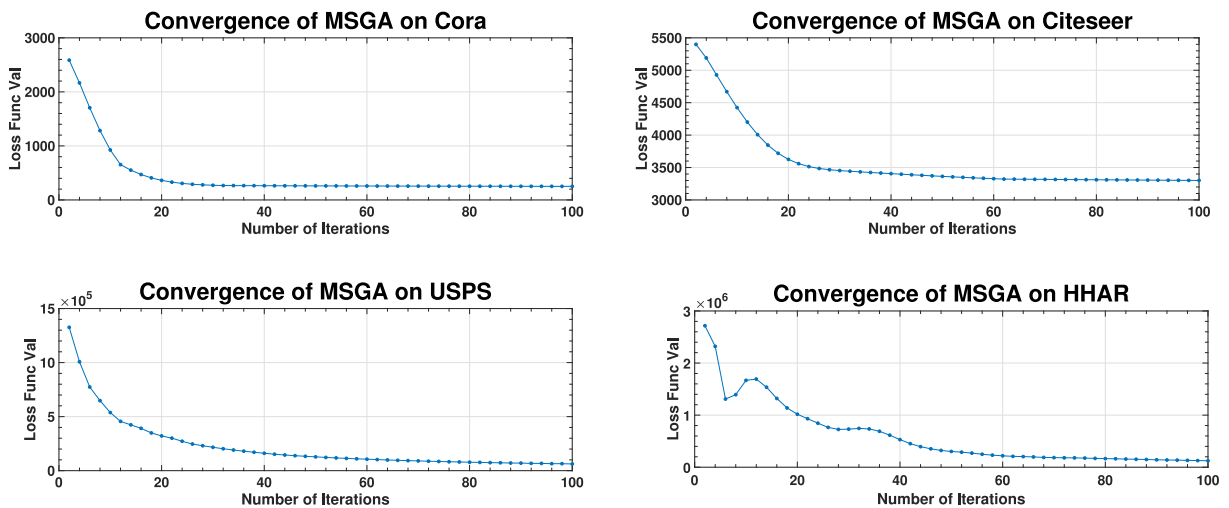


Fig. 6. Convergence curve of MSGA on four datasets.

Table 5

Ablation study on Cora and Citeseer in terms of ACC, NMI and ARI.

Loss Function			Cora			Citeseer		
$\mathcal{L}_1$	$\mathcal{L}_2$	$\mathcal{L}_3$	ACC	NMI	ARI	ACC	NMI	ARI
✗	✗	✗	0.492	0.321	0.230	0.540	0.305	0.278
✓	✗	✗	0.658	0.527	0.451	0.616	0.401	0.381
✓	✗	✓	0.688	0.539	0.472	0.646	0.408	0.388
✓	✓	✗	0.733	0.570	0.497	0.678	0.421	0.402
✓	✓	✓	0.747	0.584	0.519	0.698	0.433	0.415

Table 6

Ablation study on multi-scale strategy.

Strategy	Cora			Citeseer		
	ACC	NMI	ARI	ACC	NMI	ARI
Without multi-scale	0.714	0.545	0.469	0.666	0.408	0.393
With multi-scale	0.747	0.584	0.519	0.698	0.433	0.415

Table 7

Ablation study on attention mechanism.

Strategy	Cora			Citeseer		
	ACC	NMI	ARI	ACC	NMI	ARI
Without attention	0.658	0.511	0.448	0.613	0.346	0.332
With attention	0.747	0.584	0.519	0.698	0.433	0.415

we employ a training trick to tackle this issue. The convergence analysis experiment can prove that this adjusted training method can make MSGA converge eventually.

4.5. Ablation study

In this subsection, we conduct the ablation study on Cora and Citeseer datasets to further analyze the importance of each module in our MSGA.

As shown in Table 5, each loss corresponds to the module we proposed in this paper. For clarify, $\mathcal{L}_1$ is the loss of feature extraction module, which is consisted of node attribute reconstruction loss $\mathcal{L}_{att}$ and graph structure reconstruction loss $\mathcal{L}_{gra}$. $\mathcal{L}_2$ is the loss of multi-scale self-expression module, which contains regularization $\mathcal{L}_{reg}$ and multi-scale self-expression loss $\mathcal{L}_{mss}$, and $\mathcal{L}_3$ is the loss of self-supervised module. Specifically, we record the result of employing none of them by only using Kmeans method. The clustering performance is shown in Table 5, of which we can have the following observation:

- The multi-scale self-expression module and self-supervised module make a great contribution to the clustering performance, which cooperate well with the feature extraction module.
- Adopting the GCN-based feature extraction module can lead to better performance, which indicates learning a good feature representation $\mathbf{H}$ for clustering is of crucial importance.
- The attempt of employing the self-supervised module is meaningful, which uses the clustering labels to self-supervise the learning of node representation and help learn more current clustering information.
- Compared with the self-supervised module, the multi-scale self-expression module shows a more obvious impact, which means a proper clustering-oriented loss function plays a significant role in clustering tasks.

For further analyzing the importance of the multi-scale strategy and attention mechanism, we design the following two ablation study. First, by employing the self-expression layer from deep subspace clustering network (DSC) instead of our proposed multi-scale self-expression module, we can compare the difference between single self-expression layer and multi-scale self-expression layer. Then, we employ graph auto-encoders (GAE) which only uses GCN method without attention mechanism instead of GATE in the feature extraction module to reconstruct nodes representation. In this way, we can know the importance of attention mechanism. We adopt a controlled variable approach that the rest modules are kept the same during this two experiments and the result is shown on Tables 6 and 7, respectively.

From Table 6, we can observe that the performance with multi-scale strategy has a promotion of almost 4% on both two datasets than use only single self-expression layer, which can verify the multi-scale strategy effectively exploits the embedded information in different layers. Moreover, it also indicates the adaptive fusion method we used to integrate the self-expression matrix $\mathbf{C}$ from different layers has made a success.

From Table 7, we can observe that the method without attention mechanism achieves poorer performance, which means using attention mechanism to aggregate the information of neighbor nodes can make a huge improvement for the learning of node representation. This also confirms the previous conclusion that learning a discriminative latent representation $\mathbf{H}$ is of great importance for clustering tasks.

5. Conclusion

We have proposed a novel end-to-end framework called Multi-Scale Graph Attention Subspace Clustering Network (MSGAN), which adopts multi-scale self-expression layers to fully exploit the latent information embedded in different layers of the encoder. Moreover, by introducing the self-supervision strategy, the cluster-

ing information is effectively utilized to improve the learning of node representation. Experimental results on five datasets demonstrate that our method achieves impressive performance over other outstanding methods in terms of several metrics.

CRedit authorship contribution statement

Tong Wang: Conceptualization, Methodology, Software, Writing - original draft. **Junhua Wu:** Supervision. **Zhenquan Zhang:** Software, Investigation. **Wen Zhou:** Writing - review & editing. **Guang Chen:** Validation, Formal analysis, Writing - review & editing. **Shasha Liu:** Methodology, Supervision, Writing - review & editing.

Declaration of Competing Interest

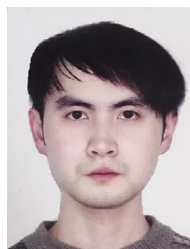
The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors would like to thank the anonymous reviewers and AE for their constructive comments and suggestions, which improved the paper substantially. This work is supported in part by the National Natural Science Foundation of China (61902197, 61802177).

References

- [1] F. Tian, B. Gao, Q. Cui, E. Chen, T.-Y. Liu, Learning deep representations for graph clustering, *AAAI* (2014) 1293–1299.
- [2] E.S. Schaeffer, Graph clustering, *Comput. Sci. Rev.* (2007) 27–64.
- [3] E. Elhamifar, R. Vidal, Sparse subspace clustering: algorithm, theory, and applications, *IEEE Trans. Pattern Anal. Mach. Intell.* (2013) 2765–2781.
- [4] M. Abavisani, M.V. Patel, Deep multimodal subspace clustering networks, *IEEE J. Sel. Top. Signal Process.* (2018) 1601–1614.
- [5] G. Liu, Z. Lin, S. Yan, J. Sun, Y. Yu, Y. Ma, Robust recovery of subspace structures by low-rank representation, *IEEE Trans. Pattern Anal. Mach. Intell.* (2013) 171–184.
- [6] F.P., V.R., R.A., A closed form solution to robust subspace estimation and clustering, *Computer Vision and Pattern Recognition* (2011) 1801–1807.
- [7] C.-Y. Lu, H. Min, Z.-Q. Zhao, L. Zhu, D.-S. Huang, S. Yan, Robust and efficient subspace segmentation via least squares regression, in: *Proceedings of the 12th European conference on Computer Vision – Volume Part VI*, 2012, pp. 347–360.
- [8] C.-G. Li, R. Vidal, Structured sparse subspace clustering: a unified optimization framework, *Comput. Vis. Pattern Recogn.* (2015) 277–286.
- [9] X. Peng, Z. Yu, Z. Yi, H. Tang, Constructing the l2 graph for robust subspace learning and subspace clustering, *IEEE Trans. Cybern.* (2016) 1053–1066.
- [10] J. Zhang, C.-G. Li, H. Zhang, J. Guo, Low rank and structured sparse subspace clustering, *IEEE International Conference on Visual Communications and Image Processing (VCIP)*.
- [11] C. You, C.-G. Li, P.D. Robinson, R. Vidal, Oracle based active set algorithm for scalable elastic net subspace clustering, *CVPR* (2016) 3928–3937.
- [12] Q. Gao, W. Xia, Z. Wan, D. Xie, P. Zhang, Tensor-svd based graph learning for multi-view subspace clustering, *AAAI* (2020) 2930–2937.
- [13] W. Xia, X. Zhang, Q. Gao, X. Shu, J. Han, X. Gao, Multi-view subspace clustering by an enhanced tensor nuclear norm, *IEEE Trans. Cybern.* doi: 10.1109/TCYB.2021.3052352..
- [14] H. Wang, G. Han, B. Zhang, G. Tao, H. Cai, Exsavi: Excavating both sample-wise and view-wise relationships to boost multi-view subspace clustering, *Neurocomputing* (2020) 66–78.
- [15] W. Xia, Q. Wang, Q. Gao, X. Zhang, X. Gao, Self-supervised graph convolutional network for multi-view clustering, *IEEE Trans. Multimedia* (2021), <https://doi.org/10.1109/TMM.2021.3094296>.
- [16] E.G. Hinton, R.R. Salakhutdinov, Reducing the dimensionality of data with neural networks, *Science*, New York, N.Y., 2006, pp. 504–507.
- [17] P. Ji, T. Zhang, H. Li, M. Salzmann, D.I. Reid, Deep subspace clustering networks, *Neural Inf. Process. Syst.* (2017) 23–32..
- [18] N.T. Kipf, M. Welling, Variational graph auto-encoders, *CoRR*.
- [19] N.T. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, *international conference on learning representations*.
- [20] S. Pan, R. Hu, G. Long, J. Jiang, L. Yao, C. Zhang, Adversarially regularized graph autoencoder for graph embedding, *IJCAI* (2018) 2609–2615.
- [21] C. Wang, S. Pan, G. Long, X. Zhu, J. Jiang, Mgae: marginalized graph autoencoder for graph clustering, *CIKM* (2017) 889–898.
- [22] A. Salehi, H. Davulcu, Graph attention auto-encoders, *CoRR*.
- [23] A. Ng, M. Jordan, Y. Weiss, On spectral clustering: analysis and an algorithm, *Adv. Neural Inf. Process. Syst.* 14 (2001) 849–856.
- [24] A. Gruber, Y. Weiss, Multibody factorization with uncertainty and missing data using the em algorithm, *CVPR* (2004) 707–714.
- [25] Q. Mo, A.B. Draper, Semi-nonnegative matrix factorization for motion segmentation with missing data, *ECCV* (7) (2012) 402–415..
- [26] G. Chen, G. Lerman, Spectral curvature clustering (scc), *Int. J. Comput. Vision* (2009) 317–330.
- [27] L. Wang, J. Huang, M. Yin, R. Cai, Z. Hao, Block diagonal representation learning for robust subspace clustering, *Inf. Sci.* (2020) 54–67.
- [28] M. Yin, S. Xie, Z. Wu, Y. Zhang, J. Gao, Subspace clustering via learning an adaptive low-rank graph, *IEEE Trans. Image Process.* (2018) 3716–3728.
- [29] E. Elhamifar, v. v. r. Vidal, Sparse subspace clustering, *Comput. Vis. Pattern Recogn.* (2009) 2790–2797..
- [30] P. Ji, M. Salzmann, H. Li, Efficient dense subspace clustering, *WACV* (2014) 461–468.
- [31] R. Vidal, P. Favaro, Low rank subspace clustering (lrscl), *Pattern Recogn. Lett.* (2014) 47–61.
- [32] M.V. Patel, V.H. Nguyen, R. Vidal, Latent space sparse subspace clustering, *Comput. Vis.* (2013) 225–232.
- [33] M.V. Patel, R. Vidal, Kernel sparse subspace clustering, *ICIP* (2014) 2849–2853.
- [34] M. Yin, Y. Guo, J. Gao, Z. He, S. Xie, Kernel sparse subspace clustering on symmetric positive definite manifolds, *CVPR* (2016) 5157–5164.
- [35] Q. Huang, Y. Zhang, H. Peng, T. Dan, W. Weng, H. Cai, Deep subspace clustering to achieve jointly latent feature extraction and discriminative learning, *Neurocomputing* (2020) 340–350.
- [36] W. Huang, M. Yin, J. Li, S. Xie, Deep clustering via weighted k-subspace network, *IEEE Signal Process. Lett.* (2019) 1628–1632.
- [37] J.A. Hartigan, M.A. Wong, Algorithm as 136: A k-means clustering algorithm, *J. R. Stat. Soc. Ser. C (Appl. Stat.)* 28(1) (1979) 100–108..
- [38] B. Perozzi, R. Al-Rfou, S. Skiena, Deepwalk: online learning of social representations, *KDD* (2014) 701–710.
- [39] S. Cao, W. Lu, Q. Xu, Deep neural networks for learning graph representations, *AAAI* (2016) 1145–1152.
- [40] C. Wang, S. Pan, R. Hu, G. Long, J. Jiang, C. Zhang, Attributed graph clustering: a deep attentional embedding approach, *IJCAI* (2019) 3670–3676.
- [41] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, *ICLR*.
- [42] S. Kou, W. Xia, X. Zhang, Q. Gao, X. Gao, Self-supervised graph convolutional clustering by preserving latent distribution, *Neurocomputing* 437 (2021) 218–226.
- [43] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, T. Eliassi-Rad, Collective classification in network data, *AI Mag.* 29 (3) (2008), 93–93.
- [44] C. Yang, Z. Liu, D. Zhao, M. Sun, Y.E. Chang, Network representation learning with rich text information, *IJCAI*.
- [45] B. Deyu, W. Xiao, S. Chuan, Z. Meiqi, L. Emiao, C. Peng, Structural deep clustering network, *WWW '20: The Web Conference 2020 Taipei Taiwan April, 2020* (2020) 1400–1410..
- [46] D. Cai, X. He, J. Han, Document clustering using locality preserving indexing, *IEEE Trans. Knowl. Data Eng.* (2005) 1624–1637.
- [47] J. unkras, Algorithms for the assignment and transportation problems, *J. Soc. Ind. Appl. Math.*.
- [48] A.P. Estévez, M. Tesmer, A.C. Perez, M.J. Zurada, Normalized mutual information feature selection, *IEEE Trans. Neural Networks* (2009) 189–201.
- [49] M.J. Santos, M. Embrechts, On the use of the adjusted rand index as a metric for evaluating supervised classification, *ICANN* (2) (2009) 175–184..
- [50] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, G.D. Murray, B. Steiner, A.P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, X. Zhang, Tensorflow: a system for large-scale machine learning, *OSDI*.
- [51] P.D. Kingma, L.J. Ba, Adam: A method for stochastic optimization, *international conference on learning representations*.



Tong Wang received the B.Eng. degree in Software Engineering from Lanzhou University of Technology, Lanzhou, China, in 2018. He is currently pursuing the master's degree in computer technology in Nanjing Tech University, Nanjing, China. His research interests include pattern recognition, machine learning and deep learning.



Junhua Wu received the Ph.D. degree from Southeast University, Nanjing, China. She is currently an associate professor with the School of Computer Science and Technology, Nanjing Tech University. Her research interests include software engineering, code generation and natural language processing.



Guang Chen is currently pursuing the master's degree in artificial intelligence in Beijing University of Posts and Telecommunications, Beijing, China. His research interests include recommended system, machine learning and deep learning.



Zhenquan Zhang is currently pursuing the master's degree in computer science and technology in Southeast University, Nanjing, China. His research interests include medical image analysis, computer vision and deep learning.



Shasha Liu is currently pursuing the master's degree in software engineering in Beijing University of Posts and Telecommunications, Beijing, China. Her research interests include pattern recognition, machine learning and deep learning.



Wen Zhou is currently pursuing the master's degree in computer technology in Nanjing Tech University, Nanjing, China. Her research interests include software engineering, code generation and natural language processing.